Lezione 4 - Flask RESTful

Introduzione

Nella **lezione precedente** abbiamo visto come implementare, in Flask, le richieste associate ai principali **metodi HTTP** (GET, POST, PUT, DELETE) utilizzando le route tradizionali.
Questo approccio è semplice ed efficace per applicazioni di piccole dimensioni, ma può diventare **poco strutturato** quando il numero di endpoint cresce e l'API diventa più complessa.

Flask, infatti, è un **micro-framework**:

- **fornisce solo gli strumenti essenziali e lascia allo sviluppatore la responsabilità di organizzare il codice.**
  Quando si sviluppano **API REST strutturate**, questo può portare a:
- **molte funzioni di route sparse nel codice;**
- **logica di validazione ripetuta;**
- **difficoltà nel mantenere uno stile uniforme delle risposte.**

Per risolvere questi problemi esiste **Flask-RESTful**, una libreria che estende Flask e introduce un modello di sviluppo **più vicino ai principi dell'architettura REST**.

Perché usare Flask-RESTful

Flask-RESTful fornisce una serie di funzionalità che aiutano a scrivere API più **ordinate, leggibili e manutenibili**.

1. Struttura orientata alle risorse

In Flask-RESTful ogni endpoint non è più una singola funzione, ma una classe che rappresenta una risorsa REST.
I metodi HTTP (GET, POST, PUT, DELETE, …) diventano metodi della classe, rendendo il codice più coerente con il concetto di *risorsa* e più facile da estendere.

2. Parsing automatico degli argomenti

La libreria mette a disposizione strumenti per analizzare e validare automaticamente i dati in ingresso (body e query parameters).

Questo evita controlli manuali ripetitivi e riduce il rischio di errori dovuti a input non validi.

3. Serializzazione controllata delle risposte

Flask-RESTful consente di definire in modo esplicito **quali campi esporre** nelle risposte JSON.
In questo modo è possibile:

- **mantenere un formato di output coerente;**
- **nascondere dati interni;**
- **controllare il tipo e la struttura delle informazioni restituite al client.**

4. Gestione degli errori integrata

La libreria fornisce meccanismi standard per la **gestione degli errori HTTP** (404, 400, 500, ecc.), semplificando la creazione di risposte di errore coerenti e semanticamente corrette.

5. Decoratori per validazione e formattazione

Attraverso appositi decoratori è possibile applicare validazione, trasformazione e formattazione dell'output in modo dichiarativo, migliorando la chiarezza del codice e separando la logica di business dalla presentazione dei dat i.

Installazione e configurazione di base

Per utilizzare Flask-RESTful è necessario installare la libreria tramite `pip`, il gestore di pacchetti di Python:

Shell

```shell
pip install flask-restful
```

Una volta completata l'installazione, Flask-RESTful si integra facilmente con un'applicazione Flask già esistente.
L'integrazione avviene attraverso l'oggetto `Api`:

- **funge da "contenitore" per tutte le risorse REST dell'applicazione.**

Inizializzazione dell'oggetto Api

```python
from flask import Flask
from flask_restful import Api

app = Flask(__name__)
api = Api(app)
```

In questo codice:

- `app = Flask(__name__)` **crea l'applicazione Flask, come visto in precedenza;**
- `Api(app)` **associa Flask-RESTful all'applicazione Flask, permettendo alla libreria di gestire le route REST.**

Differenza rispetto a Flask "classico"

Con Flask tradizionale, le route vengono definite tramite il decoratore:

```python
@app.route('/libri')
def get_libri():
    ...
```

Con Flask-RESTful, invece, **non si usano direttamente i decoratori** `@app.route`.

- **Le route vengono associate a classi che rappresentano risorse,**
- **la registrazione avviene tramite l'oggetto** `Api`**.**

Questo comporta un cambio di paradigma importante:

- **ogni risorsa REST è rappresentata da una classe;**
- i **metodi HTTP** (GET, POST, PUT, DELETE, …) diventano metodi della classe;
- **la mappatura URL ↔ risorsa è gestita centralmente dall'oggetto** `Api`**.**

✓ Vantaggi di questo approccio

L'uso dell'oggetto `Api` permette di:

- separare in modo chiaro le diverse risorse dell'API;
- organizzare il codice in maniera più modulare;

- rendere più immediata la lettura e la manutenzione del progetto;
- aderire meglio ai principi dell'architettura REST.

In sintesi, l'oggetto `Api` rappresenta **il punto di ingresso di Flask-RESTful e sostituisce l'uso diretto delle route basate su funzioni, introducendo un modello orientato alle risorse.**

Il concetto di Resource

In Flask-RESTful, il concetto centrale non è più la *route*, ma la **risorsa REST**.

- **Ogni risorsa viene rappresentata da una classe Python che eredita da `Resource`.**

Come già visto nella lezione Lezione 7 - Sistemi REST:

- **Una *risorsa* rappresenta un'entità del dominio applicativo (ad esempio: un libro, un utente, un ordine) ed è identificata da una URL.**

In una classe `Resource`:

- ogni **verbo HTTP** (`GET`, `POST`, `PUT`, `DELETE`, ...)
- corrisponde a **un metodo della classe** con lo stesso nome (`get`, `post`, `put`, `delete`).

✓ **Questo rende il codice:**

- più coerente con l'architettura REST;
- più leggibile, perché tutte le operazioni su una risorsa sono raccolte nello stesso punto;
- più facile da estendere.

Esempio: risorsa singolo libro

Python

```python
from flask_restful import Resource

class LibroResource(Resource):

    def get(self, libro_id):
```

```python
        # GET /libri/123
        return {'id': libro_id, 'titolo': 'Esempio'}

    def put(self, libro_id):
        # PUT /libri/123
        return {'messaggio': f'Libro {libro_id} aggiornato'}

    def delete(self, libro_id):
        # DELETE /libri/123
        return '', 204
```

In questo esempio:

- la **URL** (`/libri/<libro_id>`) identifica *quale risorsa* si sta manipolando;
- il **verbo HTTP** indica *che operazione* si vuole eseguire;
- ogni metodo della classe implementa una specifica azione sulla stessa risorsa.

Questo approccio separa chiaramente identità della risorsa e operazione da eseguire, che è uno dei principi fondamentali delle API REST.

Risorsa collezzione vs risorsa singola

Una buona pratica nelle API REST è distinguere tra:

- **collezione di risorse** → `/libri`
- **singola risorsa** → `/libri/<id>`

Questa separazione rende l'API più intuitiva e coerente.

Risorsa collezione

La risorsa *collezione* rappresenta:

- **l'insieme degli elementi e gestisce operazioni globali come:**
  - **recuperare la lista completa;**
  - **creare un nuovo elemento.**

Python

```python
class LibriResource(Resource):
    def get(self):
        # GET /libri
        return {'libri': []}
```

```python

    def post(self):
        # POST /libri
        return {'messaggio': 'Libro creato'}, 201

```

Significato delle operazioni:

- `GET /libri` → restituisce l'elenco dei libri;
- `POST /libri` → crea un nuovo libro nella collezione.

Risorsa singola

La risorsa *singola* rappresenta:

- **un elemento specifico ed è identificata da un ID:**
  - `GET /libri/1` → dettagli del libro con ID 1;
  - `PUT /libri/1` → aggiornamento del libro;
  - `DELETE /libri/1` → eliminazione del libro.

Questa distinzione aiuta a mantenere un'API pulita e semanticamente corretta.

Registrazione delle risorse

Come gia sappiamo le risorse, nell'architettura REST, sono associate agli endpoint.
Flask RESTful per implementare questo comportamento aggiunge una funzione built-in chiamata `add_resource`:

Python

```python
api.add_resource(LibriResource, '/libri')
api.add_resource(LibroResource, '/libri/<int:libro_id>')
```

- `LibriResource` viene collegata all'endpoint `/libri`;
- `LibroResource` viene collegata all'endpoint `/libri/<libro_id>`;
- il parametro `<int:libro_id>` viene passato automaticamente ai metodi della classe.

🔗 **Differenza rispetto a `@app.route`**

Con Flask tradizionale:

- **ogni endpoint è definito con un decoratore** `@app.route`**;**
- **la logica è distribuita in più funzioni.**

Con Flask-RESTful:

- **la registrazione avviene tramite** `add_resource`**;**
- **ogni risorsa è incapsulata in una classe;**
- **i metodi HTTP sono organizzati in modo naturale.**

In pratica, `add_resource` **sostituisce completamente l'uso di** `@app.route`, offrendo una struttura più adatta allo sviluppo di API REST complesse.

Parsing degli argomenti con `reqparse`

Flask-RESTful mette a disposizione il modulo `reqparse`:

- **che permette di gestire in modo strutturato i dati inviati dal client all'interno di una richiesta HTTP.**

In particolare, `RequestParser` consente di:

- **validare i parametri in ingresso (obbligatori, opzionali, valori ammessi);**
- **convertire automaticamente i tipi (ad esempio da stringa a intero);**
- **gestire gli errori di input in modo uniforme, restituendo risposte HTTP coerenti.**

Questo approccio evita controlli manuali sul contenuto della richiesta e rende il codice più robusto e leggibile.

Parsing dei dati nel body (POST/PUT)

Quando il client invia dati nel **body della richiesta** (tipicamente con `POST` o `PUT`), `RequestParser` permette di definirne la struttura attesa.

Python

```python
from flask_restful import Resource, reqparse

class LibriResource(Resource):
    def __init__(self):
        self.parser = reqparse.RequestParser()
```

```python

        self.parser.add_argument(
            'titolo',
            type=str,
            required=True,
            help='Titolo obbligatorio'
        )
        self.parser.add_argument(
            'autore',
            type=str,
            required=True
        )
        self.parser.add_argument(
            'anno',
            type=int
        )
        self.parser.add_argument(
            'genere',
            type=str,
            choices=['fiction', 'non-fiction']
        )

    def post(self):
        args = self.parser.parse_args()

        nuovo_libro = {
            'id': generate_id(),
            'titolo': args['titolo'],
            'autore': args['autore'],
            'anno': args.get('anno'),
            'genere': args.get('genere', 'fiction')
        }

        return nuovo_libro, 201
```

Spiegazione

- `RequestParser()`:
  - **crea un oggetto responsabile della lettura e validazione dei parametri.**
- `add_argument()`:
  - **definisce un singolo parametro:**
    - `type`: **tipo atteso del valore;**

    - `required=True` : **il parametro è obbligatorio;**
    - `help` : **messaggio di errore restituito se il parametro manca o è invalido;**
    - `choices` : **insieme di valori ammessi.**
- `parse_args()` :
    - **legge i dati dal body della richiesta;**
    - **effettua le conversioni di tipo;**
    - **genera automaticamente un errore HTTP 400 se qualcosa non è valido.**
      Se il client invia dati non conformi (ad esempio `anno` non numerico o `genere` non ammesso), Flask-RESTful risponde automaticamente con un errore chiaro e standardizzato.

Parsing dei query parameters

I **query parameters** sono parametri opzionali che:

- **il client inserisce direttamente nell'URL per modificare il comportamento della richiesta, senza cambiare la risorsa richiesta**

Esempi tipici sono:

- `limit` → **numero massimo di elementi restituiti**
- `offset` / `page` → **paginazione**
- `sort` , `order` → **ordinamento**
- `filter` → **filtri di ricerca**

Esempio di richiesta:

HTTP

```
GET /libri?limit=5&offset=10
```

In questo caso:

- la risorsa resta `/libri` ;
- i parametri servono solo a **controllare la modalità di restituzione dei dati**.

Perché usare `reqparse` anche per i query parameters

Senza `reqparse`, i parametri di query vengono letti manualmente da `request.args`, rendendo necessario gestire esplicitamente:

- la **presenza o assenza del parametro**;
- la **conversione del tipo** (ad esempio da `str` a `int`);
- la **gestione di valori non validi**.

Questo approccio porta facilmente a codice ripetitivo, poco leggibile e più soggetto a errori.

Utilizzando `RequestParser`, invece, **la struttura dei parametri attesi viene dichiarata in modo esplicito**, rendendo il metodo `get` più pulito, prevedibile e coerente.

Python

```python
def get(self):
    parser = reqparse.RequestParser()
    parser.add_argument(
        'limit',
        type=int,
        default=10,
        location='args'
    )
    parser.add_argument(
        'offset',
        type=int,
        default=0,
        location='args'
    )

    args = parser.parse_args()

    return {
        'libri': [],
        'limit': args['limit'],
        'offset': args['offset']
    }
```

Spiegazione

- `location='args'` **indica che il parametro deve essere letto dalla query string.**
- `default` **fornisce un valore di fallback se il parametro non è presente.**
- Anche in questo caso:
  - **i tipi vengono convertiti automaticamente;**
  - **eventuali valori non validi generano errori standardizzati (HTTP 400).**

> ✓ **Vantaggi dell'uso di** `reqparse`
>
> L'utilizzo di `RequestParser` offre diversi benefici:
>
> - **validazione automatica dell'input**;
> - **gestione coerente degli errori**;
> - **riduzione del codice di controllo manuale**;
> - **maggiore chiarezza nella definizione dell'API**.

In sintesi, `reqparse` aiuta a definire in modo esplicito il "contratto" tra client e server, rendendo l'API REST più affidabile e manutenibile.

Marshalling e serializzazione dell'output

Uno degli aspetti più delicati nella progettazione di un'API REST è

- **decidere come esporre i dati verso il client**.
  Restituire direttamente oggetti Python o dizionari "grezzi" può portare a:
- **risposte incoerenti;**
- **esposizione involontaria di dati interni;**
- **difficoltà nel mantenere stabile il formato dell'API nel tempo.**

Flask-RESTful affronta questo problema introducendo il concetto di **marshalling:**

- ovvero la **trasformazione controllata dei dati** in un formato JSON ben definito, tramite `fields` e il decoratore `@marshal_with`.

Definizione della struttura di output

La struttura dell'output viene dichiarata tramite un dizionario di `fields`:

- **descrive esattamente quali attributi devono comparire nella risposta e in che formato.**

Python

```python
from flask_restful import fields

libro_fields = {
    'id': fields.Integer,
    'titolo': fields.String,
    'autore': fields.String,
    'isbn': fields.String,
    'anno_pubblicazione': fields.Integer,
    'url': fields.Url('libro_detail')
}
```

In questo modo:

- ogni campo dell'output è **esplicitamente dichiarato**;
- il tipo di dato restituito è **controllato e coerente**;
- è possibile aggiungere **link ipermediali** (HATEOAS) tramite `fields.Url`.

⚠ `fields` **non è un metodo che "chiama" l'attributo, ma un insieme di descrittori di serializzazione.**

In pratica per ogni voce del dizionario:

Python

```python
'id': fields.Integer
```

significa:

> "Quando esponi questa risorsa verso il client, prendi l'attributo `id` dell'oggetto Python e serializzalo come `Integer` nel JSON di risposta".

Quindi `fields` è una sorta di **schema di rappresentazione della risorsa**, non un accesso diretto ai dati.

Questo approccio consente di:

- **esporre solo i campi desiderati;**

- **nascondere attributi interni o sensibili;**
- **garantire uniformità e stabilità nelle risposte dell'API.**

Uso del decoratore `marshal_with`

Una volta definita la struttura, è possibile applicarla automaticamente alle risposte utilizzando il decoratore `@marshal_with`.

> Il decoratore `@marshal_with` **non definisce le chiamate HTTP**, **ma definisce come viene costruita la risposta HTTP.**

Ad esempio:

```python
@marshal_with(libro_fields)
def get(self, libro_id):
    ...
```

Questo significa:

> **Qualunque valore venga restituito da questo metodo, trasformalo secondo lo schema `libro_fields` prima di inviarlo al client.**

Quindi `@marshal_with`:

- **intercetta il valore di ritorno del metodo (`return libro`);**
- **applica la struttura definita nei `fields`;**
- **genera automaticamente il JSON finale della response.**

🔗 Relazione con i metodi HTTP

Hai colto bene questo punto, ma lo rifiniamo:

- i **metodi della classe** (`get`, `post`, `put`, `delete`) rappresentano i **verbi HTTP**;
- `@marshal_with` **non gestisce il routing, ma la serializzazione della risposta prodotta da quei metodi.**

✓ **Quindi è più corretto dire che:**

`@marshal_with` definisce:

- **il formato delle risposte HTTP dei metodi della `Resource`, non la chiamata HTTP in sé.**

Esempio:

Python

```python
from flask_restful import marshal_with, abort

class LibroResource(Resource):

    @marshal_with(libro_fields)
    def get(self, libro_id):
        libro = get_libro_from_db(libro_id)
        if not libro:
            abort(404)
        return libro
```

Con questo meccanismo, Flask-RESTful:

- **riceve un oggetto Python (ad esempio un'istanza o un dizionario);**
- **applica la struttura definita nei `fields`;**
- **genera automaticamente una risposta JSON conforme allo schema dichiarato.**

Il metodo `get` può quindi concentrarsi esclusivamente sulla **logica applicativa**, senza preoccuparsi della formattazione dell'output.

✓ **Vantaggi del marshalling**

L'uso di `fields` e `@marshal_with` porta numerosi benefici:

- **separazione chiara tra logica di business e rappresentazione dei dati;**
- **maggiore leggibilità del codice;**
- **risposte prevedibili e standardizzate;**
- **facilità di manutenzione e di evoluzione dell'API.**

In sintesi

Il marshalling in Flask-RESTful permette di definire in modo esplicito **come una risorsa deve essere rappresentata verso l'esterno**, trasformando automaticamente gli oggetti interni in JSON coerenti e controllati.

Questo rende l'API più **robusta, sicura e aderente ai principi REST**.

Esempio di risposta strutturata

In questo esempio viene mostrato come **costruire una risposta JSON complessa e strutturata**, tipica di una API REST che restituisce **liste paginate di risorse**.

L'obiettivo è restituire al client non solo i dati principali (i libri), ma anche **informazioni di contesto**, come il numero totale di elementi e la pagina corrente.

1. **Definizione della struttura di output**

Python

```python
libri_fields = {
    'libri': fields.List(fields.Nested(libro_fields)),
    'totale': fields.Integer,
    'pagina': fields.Integer
}
```

Questo dizionario definisce **lo schema della risposta HTTP**.

- `libri`
  - È una **lista** (`fields.List`)
  - Ogni elemento della lista è una **struttura annidata** (`fields.Nested`)
  - La struttura di ciascun libro è definita da `libro_fields`

> **In altre parole:**
>
> la chiave `libri` conterrà un array di oggetti libro, ciascuno serializzato secondo lo schema `libro_fields`.

- `totale`
  - **È un intero (`fields.Integer`)**
  - **Rappresenta il numero totale di libri disponibili, indipendentemente dalla paginazione**

- `pagina`
  - **È un intero (`fields.Integer`)**
  - **Indica la pagina corrente richiesta dal client**

Questa struttura consente di **standardizzare le risposte** e di fornire informazioni utili per la navigazione lato client.

2. **Metodo `get` della risorsa `LibriResource`**

Python

```python
class LibriResource(Resource):
    @marshal_with(libri_fields)
    def get(self):
```

La classe `LibriResource` rappresenta la **risorsa collezione** `/libri`.

Il decoratore `@marshal_with(libri_fields)` indica che:

> Il valore restituito dal metodo `get` deve essere **serializzato secondo la struttura definita in** `libri_fields` prima di essere inviato al client.

3. **Parsing dei parametri di paginazione**

Python

```python
parser = reqparse.RequestParser()
parser.add_argument('page', type=int, default=1)
args = parser.parse_args()
```

Qui viene gestito il parametro di query `page`, che consente al client di richiedere una pagina specifica.

- `type=int` → **converte automaticamente il valore in intero**
- `default=1` → **se il client non specifica il parametro, viene usata la prima pagina**

Esempio di richiesta:

HTTP

```http
GET /libri?page=2
```

4. **Costruzione della risposta logica**

```python
return {
    'libri': get_libri_paginated(args['page']),
    'totale': count_libri(),
    'pagina': args['page']
}
```

Il metodo restituisce **un dizionario Python**, non direttamente JSON.

- `get_libri_paginated(args['page'])`
  - **Recupera solo i libri della pagina richiesta**
- `count_libri()`
  - **Restituisce il numero totale dei libri presenti**
- `args['page']`
  - **Riporta la pagina corrente nella risposta**

Codice completo

```python
from flask_restful import Resource, reqparse, fields, marshal_with, abort

# Definizione dei campi per un singolo libro
libro_fields = {
    'id': fields.Integer,
    'titolo': fields.String,
    'autore': fields.String,
    'isbn': fields.String,
    'anno_pubblicazione': fields.Integer,
    'url': fields.Url('libro_detail')  # link automatico alla risorsa singola
}

# Definizione dei campi per la risposta della collezione di libri
libri_fields = {
    'libri': fields.List(fields.Nested(libro_fields)),  # lista di libri
    'totale': fields.Integer,                           # totale libri disponibili
```

```python
    'pagina': fields.Integer                                    # pagina richiesta
}

class LibriResource(Resource):
    @marshal_with(libri_fields)
    def get(self):
        # Parser per query parameters
        parser = reqparse.RequestParser()
        parser.add_argument('page', type=int, default=1)
        args = parser.parse_args()

        # Recupero logico dei dati
        libri_paginated = get_libri_paginated(args['page'])
        totale_libri = count_libri()
        pagina_corrente = args['page']

        # Restituisco il dizionario, marshal_with gestirà la serializzazione
        return {
            'libri': libri_paginated,
            'totale': totale_libri,
            'pagina': pagina_corrente
        }

# Funzioni di esempio (da implementare secondo la tua logica)
def get_libri_paginated(page: int):
    # Esempio: recupera i libri della pagina richiesta
    # Sostituire con accesso reale al database
    return [
        {'id': 1, 'titolo': 'Il Signore degli Anelli', 'autore': 'J.R.R. Tolkien', 'isbn': '123', 'anno_pubblicazione': 1954},
        {'id': 2, 'titolo': '1984', 'autore': 'George Orwell', 'isbn': '456', 'anno_pubblicazione': 1949}
    ]

def count_libri():
    # Esempio: ritorna il totale dei libri
    return 42

```

Test dell’API

Dopo aver implementato le risorse con Flask-RESTful, possiamo **verificare il funzionamento dell'API** utilizzando strumenti come **Postman**, **curl** o il browser (per le richieste GET).

1. **Ottenere tutti i libri**
   - Per recuperare la lista completa delle risorse della collezione:

Shell

```
GET http://localhost:5000/libri
```

- Questo richiama il metodo `get` della risorsa `LibriResource`.
- Possiamo usare query parameters per modificare la risposta, ad esempio `?page=2&limit=10` per paginare i risultati.
- La risposta sarà un JSON strutturato secondo `libri_fields`.

2. **Ottenere un singolo libro**
   - Per leggere i dettagli di un libro specifico:

Shell

```
GET http://localhost:5000/libri/1
```

- Qui chiamiamo `get` della risorsa `LibroResource`, passando l'ID del libro (`1`).
- Il server restituirà un JSON con le informazioni del libro selezionato, serializzate secondo `libro_fields`.
- Se il libro non esiste, Flask-RESTful può restituire automaticamente un errore `404 Not Found`.

3. **Creare un nuovo libro**
   - Per aggiungere un nuovo libro nella collezione, si utilizza **POST** con i dati nel body in formato JSON:

Shell

```
POST http://localhost:5000/libri
Content-Type: application/json
{
"titolo": "Il Signore degli Anelli",
"autore": "J.R.R. Tolkien",
```

```
"anno": 1954
}
```

- La risorsa `LibriResource` leggerà e **validerà i dati** usando `reqparse`.
- Se i campi obbligatori mancano o sono del tipo sbagliato, Flask-RESTful genera un errore chiaro.
- Se la creazione ha successo, il server restituisce il libro appena creato con status `201 Created`.
- **Aggiornare un libro esistente**
  - Per modificare i dati di un libro:

Shell

```
PUT http://localhost:5000/libri/1
```

- Qui chiamiamo `put` della risorsa `LibroResource`.
- Il server aggiorna solo i campi forniti nel body.
- Se il libro non esiste, possiamo gestire l'errore con `abort(404)`.

4. **DELETE**
   - Per rimuovere una risorsa:

Shell

```
DELETE http://localhost:5000/libri/2
```

- Chiama il metodo `delete` della risorsa `LibroResource`.
- Se la cancellazione ha successo, il server restituisce `204 No Content`.
- Se il libro non esiste, si restituisce un errore `404 Not Found`.